



JavaScript: A Crash Course

Part I: Core Language Syntax

Originals of Slides and Source Code for Examples:
<http://courses.coreservlets.com/Course-Materials/ajax.html>

Customized Java EE Training: <http://courses.coreservlets.com/>
Servlets, JSP, Struts, JSF/MyFaces/Facelets, Ajax, GWT, Java 5 or 6, etc. Spring/Hibernate coming soon.
Developed and taught by well-known author and developer. At public venues or onsite at *your* location.



For live Ajax & GWT training, see training courses at <http://courses.coreservlets.com/>.



Taught by the author of *Core Servlets and JSP*, *More Servlets and JSP*, and this tutorial. Available at public venues, or customized versions can be held on-site at your organization.

- Courses developed and taught by Marty Hall
 - Java 5, Java 6, intermediate/beginning servlets/JSP, advanced servlets/JSP, Struts, JSF, Ajax, GWT, custom mix of topics
- Courses developed and taught by coreservlets.com experts (edited by Marty)
 - Spring, Hibernate, EJB3, Ruby/Rails

Contact hall@coreservlets.com for details

Topics in This Section

- Overview
- JavaScript references
- Embedding in browser
- Basic syntax
- Strings and regular expressions
- Functions
- Objects

4

Java EE training: <http://courses.coreservlets.com>

© 2008 Marty Hall



Intro

Customized Java EE Training: <http://courses.coreservlets.com/>

Servlets, JSP, Struts, JSF/MyFaces/Facelets, Ajax, GWT, Java 5 or 6, etc. Spring/Hibernate coming soon.

Developed and taught by well-known author and developer. At public venues or onsite at *your* location.

Overview

- **Goals**

- Aimed at developers that already know Java
- Aimed at use of JavaScript in a Web browser
 - Especially main capabilities for Ajax developers
- Not a complete reference

- **Prerequisites**

- Familiarity with XHTML.
 - Since goal is use of JavaScript for Ajax, pages will use XHTML syntax.
- Familiarity with Java
 - Will focus on major differences from Java
- Other tutorials
 - Separate tutorials on Java, XHTML, CSS, and many Ajax-related topics can be found at coreservlets.com

6

Java EE training: <http://courses.coreservlets.com>

Firebug

- **Tutorial assumes you have Firebug**

- Firefox environment for executing and debugging JavaScript, tracing Ajax HTTP calls, exploring DOM, viewing CSS, and more
- Indispensable Ajax and JavaScript tool

- **Quick summary**

- Download from <http://getfirebug.com/>
- Invoke with F12
- Disable by default
 - Right click on green Firebug arrow at bottom right
 - Choose "Disable Firebug"
 - Right click on Firebug icon again
 - Choose "Allowed Sites"
 - Enter localhost plus your main deployment sites
 - On any site, you can still hit F12 and enable Firebug on that site (temporarily or permanently)

- **For more details**

- See "Ajax Development Tools" in coreservlets.com Ajax tutorial

7

Java EE training: <http://courses.coreservlets.com>

References

- **Books**

- *JavaScript the Definitive Guide* by David Flanagan
- *Pro JavaScript Techniques* by John Resig

- **Online References**

- JavaScript tutorial
 - <http://www.w3schools.com/js/>
- JavaScript API references (all builtin objects)
 - <http://www.w3schools.com/jsref/>
 - <http://www.devguru.com/technologies/ecmascript/QuickRef/>
 - <http://www.devguru.com/technologies/JavaScript/>
 - <http://www.javascriptkit.com/jsref/>
- HTML DOM reference (with JavaScript Examples)
 - http://www.w3schools.com/html/dom/dom_reference.asp
- Official ECMAScript specification
 - <http://www.ecma-international.org/publications/standards/Ecma-262.htm>



Embedding JavaScript in HTML

Customized Java EE Training: <http://courses.coreservlets.com/>
Servlets, JSP, Struts, JSF/MyFaces/Facelets, Ajax, GWT, Java 5 or 6, etc. Spring/Hibernate coming soon.
Developed and taught by well-known author and developer. At public venues or onsite at *your* location.

Loading Scripts

- **script with src**
 - `<script src="my-script.js" type="text/javascript"></script>`
 - Purpose
 - To define functions, objects, and variables.
 - Functions will later be triggered by buttons, other user events, inline script tags with body content, etc.
- **script with body content**
 - `<script type="text/javascript">JavaScript code</script>`
 - Purpose
 - To directly invoke code that will run as page loads
 - E.g., to output HTML content built by JavaScript
 - Don't use this approach for defining functions or for doing things that could be done in external files.
 - Slower (no browser caching) and less reusable

Example (phish.js)

```
function getMessage() {
    var amount = Math.round(Math.random() * 100000);
    var message =
        "You won $" + amount + "!\n" +
        "To collect your winnings, send your credit card\n" +
        "and bank details to oil-minister@phisher.com.";
    return(message);
}

function showWinnings1() {
    alert(getMessage());
}

function showWinnings2() {
    document.write("<h1><blink>" + getMessage() +
        "</blink></h1>");
}
```

12

Java EE training: <http://courses.coreservlets.com>

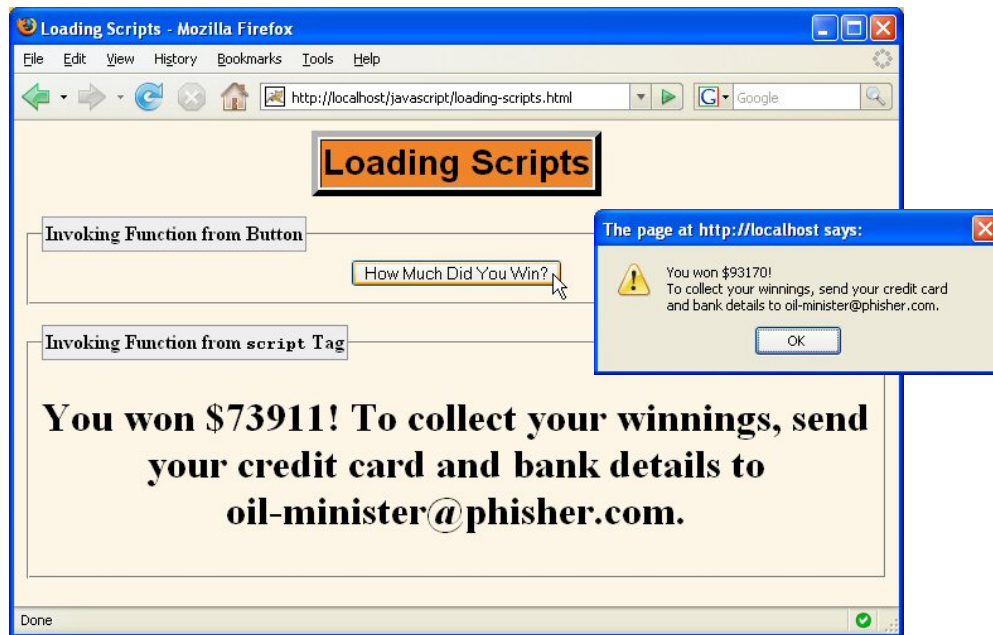
Example (loading-scripts.html)

```
<!DOCTYPE ...><html xmlns="http://www.w3.org/1999/xhtml">
<head><title>Loading Scripts</title>
...
<script src="../scripts/phish.js"
    type="text/javascript"></script>
</head>
<body>
...
    <input type="button" value="How Much Did You Win?"
        onclick='showWinnings1()' />
...
    <script type="text/javascript">showWinnings2(</script>
...
</body></html>
```

13

Java EE training: <http://courses.coreservlets.com>

Example (Results)



14

Java EE training: <http://courses.coreservlets.com>

Loading Scripts: Special Cases

- **Internet Explorer bug**
 - Scripts with src fail to load if you use `<script.../>`.
 - You must use `<script src="..." ...></script>`
- **XHTML: Scripts with body content**
 - It is an error if the body of the script contains special XML characters such as `&` or `<`
 - E.g. `<script...>if (a<b) { this(); } else { that(); }</script>`
 - So, use CDATA section unless body content is simple and clearly has no special characters
 - `<script type="text/javascript"><![CDATA[
JavaScript Code
]]></script>`

15

Java EE training: <http://courses.coreservlets.com>



Basic Syntax

Customized Java EE Training: <http://courses.coreservlets.com/>

Servlets, JSP, Struts, JSF/MyFaces/Facelets, Ajax, GWT, Java 5 or 6, etc. Spring/Hibernate coming soon.
Developed and taught by well-known author and developer. At public venues or onsite at *your* location.

Variables

- **Introduce with "var"**
 - For global variables (!) and local variables.
 - No "var" for function arguments
- **You do not declare types**
 - Some people say JavaScript is "untyped" language, but really it is "dynamically typed" language
 - JavaScript is very liberal about converting types
- **There are only two scopes**
 - Global scope
 - Be very careful with this when using Ajax. Can cause race conditions.
 - Function (lexical) scope
 - There is *not* block scope as in Java

Operators and Statements

- **Almost same set of operators as Java**
 - + (addition and String concatenation), -, *, /
 - &&, ||, ++, --, etc
 - The == comparison is more akin to Java's "equals"
 - The === operator (less used) is like Java's ==
- **Statements**
 - Semicolons are technically optional
 - But highly recommended
 - Consider
 - return x
 - return
x
 - They are not identical! The second one returns, then evaluates x. Act as though semicolons are required as in Java.
- **Comments**
 - Same as in Java (/* ... */ and // ...)

18

Java EE training: <http://courses.coreservlets.com>

Conditionals and Simple Loops

- **if/else**
 - Almost identical to Java except test can be converted to true/false instead of strict true/false
 - 0 is false, 1 is true
 - Many people avoid this and use strict booleans
- **Basic for loop**
 - Identical to Java except for variable declarations
 - for(var i=0; i<someVal; i++) { doLoopBody(); }
- **while loop**
 - Same as Java except test can be converted to boolean
 - while(someTest) { doLoopBody(); }
- **do/while loop**
 - Same as Java except test can be converted to boolean

19

Java EE training: <http://courses.coreservlets.com>

Array Basics

- **One-step array allocation**
 - `var primes = [2, 3, 5, 7, 11, 13];`
 - `var names = ["Joe", "Jane", "John", "Juan"];`
- **Two-step array allocation**
 - `var names = new Array(4);`
`names[0] = "Joe";`
`...`
`names[3] = "Juan";`
- **Indexed at 0 as in Java**
 - `for(var i=0; i<names.length; i++) {`
`doSomethingWith(names[i]);`
`}`

20

Java EE training: <http://courses.coreservlets.com>

Other Conditionals and Loops

- **switch**
 - Differs from Java in two ways
 - The "case" can be an expression
 - Values need not be ints (compared with `==`)
- **for/in loop**
 - Similar to Java for/each loop, but
 - For arrays, values are array indexes, not array values
 - Indexes are treated as strings ("0")
 - Shows only indexes with values
 - For objects, values are the property names
 - `var names = ["Joe", "Jane", "John", "Juan"];`
`for(var i in names) {`
`doSomethingWith(names[i]);`
`}`

21

Java EE training: <http://courses.coreservlets.com>

More on Arrays

- **Arrays can be sparse**
 - `var names = new Array();`
`names[0] = "Joe";`
`names[100000] = "Juan";`
- **Arrays can be resized**
 - Regardless of how arrays is created, you can do:
 - `myArray.length = someNewLength;`
 - `myArray[anyNumber] = someNewValue;`
 - This is legal regardless of which way myArray was made
- **Arrays have methods**
 - `join`, `reverse`, `sort`, `concat`, `slice`, `splice`, `toString`, etc.
 - See API reference
- **Regular objects can be treated like arrays**
 - You can use numbers (indexes) as properties

22

Java EE training: <http://courses.coreservlets.com>

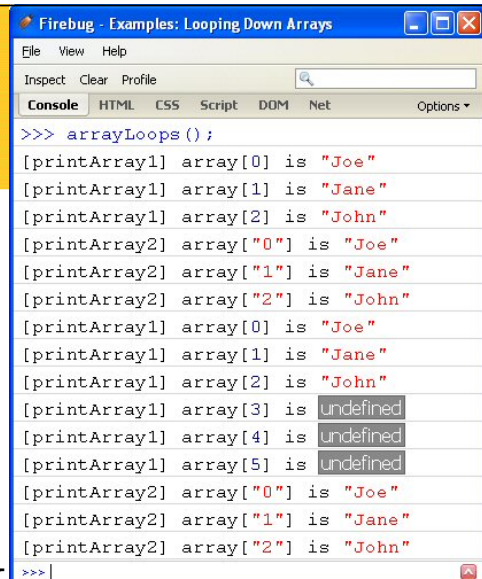
Arrays Example

```
function arrayLoops() {
    var names =
        ["Joe", "Jane",
         "John", "Juan"];
    printArray1(names);
    printArray2(names);
    names.length = 10;
    printArray1(names);
    printArray2(names);
}

function printArray1(array) {
    for(var i=0; i<array.length; i++) {
        console.log("[printArray1] array[%0] is %0", i, array[i]);
    }
}

function printArray2(array) {
    for(var i in array) {
        console.log("[printArray2] array[%0] is %0", i, array[i]);
    }
}

arrayLoops();
```



`console.log` is a printf-like way to print output in Firebug Console window. For testing/debugging only.

Direct call for interactive testing in Firebug console.
(Cut/paste all code into console command line.)

23

Java EE training: <http://courses.coreservlets.com>

The Math Class

- **Almost identical to Java**
 - Like Java, static methods (Math.cos, Math.random, etc.)
 - Like Java, logs are base e, trig functions are in radians
- **Functions**
 - Math.abs, Math.acos, Math.asin, Math.atan, Math.atan2, Math.ceil, Math.cos, Math.exp, Math.floor, Math.log, Math.max, Math.min, Math.pow, Math.random, Math.round, Math.sin, Math.sqrt, Math.tan
- **Constants**
 - Math.E, Math.LN10, Math.LN2, Math.LOG10E, Math.PI, Math.SQRT1_2, Math.SQRT2



Strings and Regular Expressions

Basics

- **You can use double or single quotes**
 - `var names = ["Joe", 'Jane', "John", 'Juan'];`
- **You can access length property**
 - E.g., `"foobar".length` returns 6
- **Numbers can be converted to strings**
 - Automatic conversion during concatenations.
String need not be first as in Java
 - `var val = 3 + "abc" + 5; // Result is "3abc5"`
 - Conversion with fixed precision
 - `var n = 123.4567;`
`var val = n.toFixed(2); // Result is 123.46 (not 123.45)`
- **Strings can be compared with ==**
 - `"foo" == 'foo'` returns true
- **Strings can be converted to numbers**
 - `var i = parseInt("37 blah"); // Result is 37 – ignores blah`
 - `var d = parseFloat("6.02 blah"); // Ignores blah`

26

Java EE training: <http://courses.coreservlets.com>

Core String Methods

- **Simple methods similar to Java**
 - `charAt`, `indexOf`, `lastIndexOf`, `substring`, `toLowerCase`, `toUpperCase`
- **Methods that use regular expressions**
 - `match`, `replace`, `search`, `split`
- **HTML methods**
 - `anchor`, `big`, `bold`, `fixed`, `fontcolor`, `fontsize`, `italics`, `link`, `small`, `strike`, `sub`, `sup`
 - `"test".bold().italics().fontcolor("red")` returns
`'<font color="red"><i><b>test</b></i></font>'`
 - These are technically nonstandard methods, but supported in all major browsers
 - But I prefer to construct HTML strings explicitly anyhow

27

Java EE training: <http://courses.coreservlets.com>

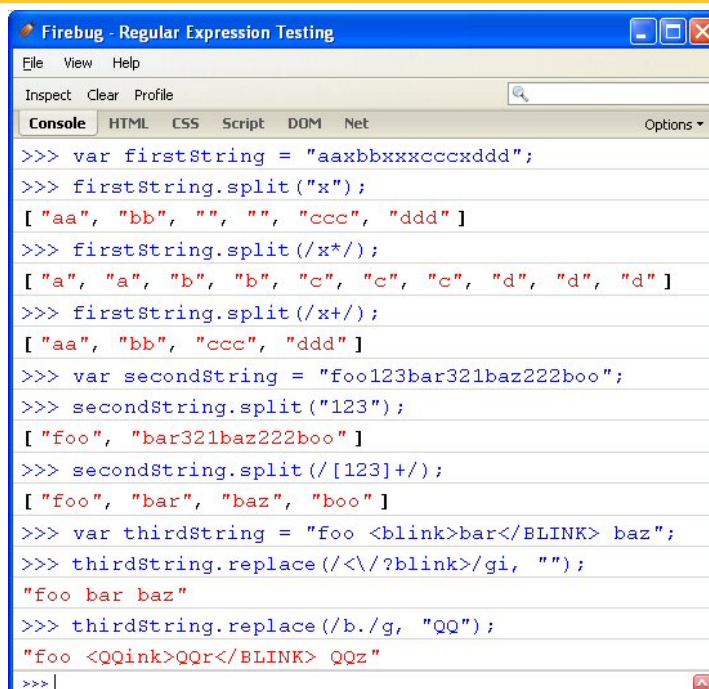
Regular Expressions

- **You specify a regexp with /pattern/**
 - Not with a String as in Java
- **Most special characters same as in Java**
 - `^`, `$`, `.` – beginning, end of string, any one char
 - `\` – escape what would otherwise be a special character
 - `*`, `+`, `?` – 0 or more, 1 or more, 0 or 1 occurrences
 - `{n}`, `{n,}` – exactly n, n or more occurrences
 - `[]` – grouping
 - `\s`, `\S` – whitespace, non-whitespace
 - `\w`, `\W` – word char (letter or number), non-word char
- **Modifiers**
 - `/pattern/g` – do global matching (find all matches, not just first one)
 - `/pattern/i` – do case-insensitive matching
 - `/pattern/m` – do multiline matching

28

Java EE training: <http://courses.coreservlets.com>

Regular Expression: Examples



```
File View Help
Inspect Clear Profile
Console HTML CSS Script DOM Net Options
>>> var firstString = "aaxbbxxxcccxd";
>>> firstString.split("x");
[ "aa", "bb", "", "", "ccc", "dd" ]
>>> firstString.split(/x*/);
[ "a", "a", "b", "b", "c", "c", "c", "d", "d", "d" ]
>>> firstString.split(/x+/);
[ "aa", "bb", "ccc", "dd" ]
>>> var secondString = "foo123bar321baz222boo";
>>> secondString.split("123");
[ "foo", "bar321baz222boo" ]
>>> secondString.split(/[123]+/);
[ "foo", "bar", "baz", "boo" ]
>>> var thirdString = "foo <blink>bar</BLINK> baz";
>>> thirdString.replace(/<\/?blink>/gi, "");
"foo bar baz"
>>> thirdString.replace(/b./g, "QQ");
"foo <QQink>QQr</BLINK> QQz"
>>>
```

29

Java EE training: <http://courses.coreservlets.com>

More Information on Regular Expressions

- **Online API references given earlier (See RegExp class)**
 - http://www.w3schools.com/jsref/jsref_obj_regexp.asp
 - <http://www.devguru.com/technologies/ecmascript/QuickRef/regexp.html>
- **JavaScript Regular Expression Tutorials**
 - http://www.evolt.org/article/Regular_Expressions_in_JavaScript/17/36435/
 - <http://www.javascriptkit.com/javatutors/re.shtml>



Functions

Customized Java EE Training: <http://courses.coreservlets.com/>

Servlets, JSP, Struts, JSF/MyFaces/Facelets, Ajax, GWT, Java 5 or 6, etc. Spring/Hibernate coming soon.

Developed and taught by well-known author and developer. At public venues or onsite at *your* location.

Overview

- **Not similar to Java**
 - JavaScript functions very different from Java methods
- **Main differences from Java**
 - You can have global functions
 - Not just methods (functions as part of objects)
 - You don't declare return types or argument types
 - Caller can supply any number of arguments
 - Regardless of how many arguments you defined
 - Functions are first-class datatypes
 - You can pass functions around, store them in arrays, etc.
 - You can create anonymous functions (closures)
 - Critical for Ajax
 - These are equivalent
 - `function foo(...) {...}`
 - `var foo = function(...) {...}`

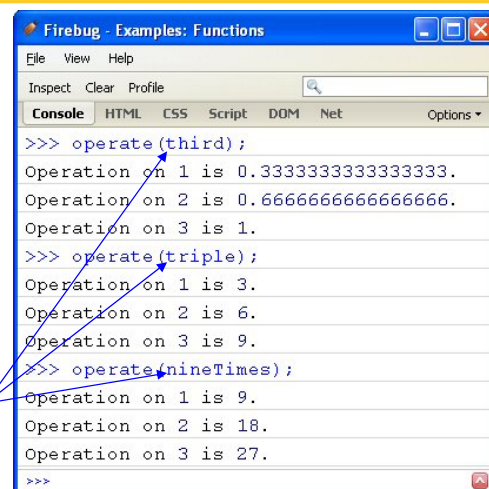
32

Java EE training: <http://courses.coreservlets.com>

Passing Functions: Example

```
function third(x) {  
    return(x / 3);  
}  
  
function triple(x) {  
    return(x * 3);  
}  
  
function nineTimes(x) {  
    return(x * 9);  
}  
  
function operate(f) {  
    var nums = [1, 2, 3];  
    for(var i=0; i<nums.length; i++) {  
        var num = nums[i];  
        console.log("Operation on %o is %o.",  
                    num, f(num));  
    }  
}
```

Function as argument.



33

Java EE training: <http://courses.coreservlets.com>

Anonymous Functions

- **Anonymous functions (or closures) let you capture local variables inside a function**
 - You can't do Ajax without this!
- **Basic anonymous function**
 - `operate(function(x) { return(x * 20); });`
 - Outputs 20, 40, 60
 - The "operate" function defined on previous page
- **Anonymous function with captured data**
 - ```
function someFunction(args) {
 var val = someCalculation(args);
 return(function(moreArgs) {
 doSomethingWith(val, moreArgs);
 });
}
var f1 = someFunction(args1);
var f2 = someFunction(args2);
f1(args3); // Uses one copy of "val"
f2(args3); // Uses a different copy of "val"
```

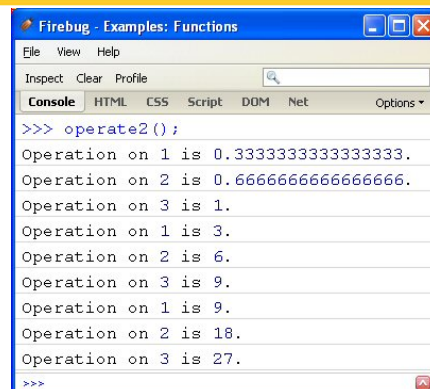
34

Java EE training: <http://courses.coreservlets.com>

## Anonymous Functions: Example

```
function multiplier(m) {
 return(function(x)
 { return(x * m); });
}
```

```
function operate2() {
 var nums = [1, 2, 3];
 var functions =
 [multiplier(1/3), multiplier(3), multiplier(9)];
 for(var i=0; i<functions.length; i++) {
 for(var j=0; j<nums.length; j++) {
 var f = functions[i];
 var num = nums[j];
 console.log("Operation on %o is %o.",
 num, f(num));
 }
 }
}
```



35

Java EE training: <http://courses.coreservlets.com>

# Optional Args and Varargs

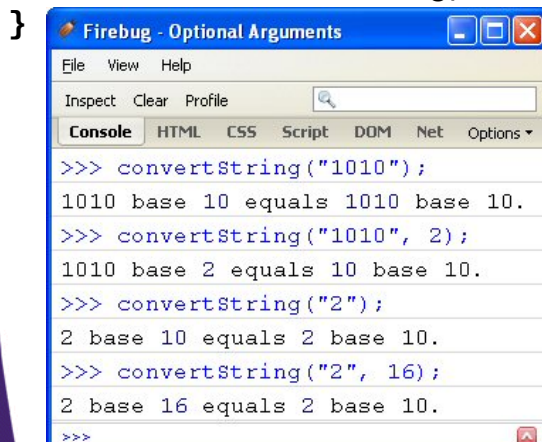
- You can call any function with any number of arguments
  - If called with fewer args, extra args equal "undefined"
    - You can use `typeof arg == "undefined"` for this
      - You can also use boolean comparison if you are sure that no real value could match (e.g., 0 and undefined both return true for `!arg`)
    - Use comments to indicate optional args
      - `function foo(arg1, arg2, /* Optional */ arg3) {...}`
  - If called with extra args, you can use "arguments" array
    - Regardless of defined variables, `arguments.length` tells you how many arguments were supplied, and `arguments[i]` returns the designated argument
    - Use comments to indicate extra args
      - `function bar(arg1, arg2 /* varargs */) { ... }`

36

Java EE training: <http://courses.coreservlets.com>

# Optional Arguments

```
function convertString(numString, /* Optional */ base) {
 if (typeof base == "undefined") {
 base = 10;
 }
 var num = parseInt(numString, base);
 console.log("%s base %o equals %o base 10.",
 numString, base, num);
}
```



37

Java EE training: <http://courses.coreservlets.com>



# Varargs

```
function longestString(/* varargs */) {
 var longest = "";
 for(var i=0; i<arguments.length; i++) {
 var candidateString = arguments[i];
 if (candidateString.length > longest.length) {
 longest = candidateString;
 }
 }
 return(longest);
}

longestString("a", "bb", "ccc", "dddd");
 // Returns "dddd"
```



# Objects

# Basics

- **Constructors**

- Functions named for class names. Then use "new".
  - No separate class definition! No "real" OOP in JavaScript!
- Can define properties with "this"
  - You must use "this" for properties used in constructors  
`function MyClass(n1) { this.foo = n1; }`  
`var m = new MyClass(10);`

- **Properties (instance variables)**

- You don't define them separately
  - Whenever you refer to one, JavaScript just creates it  
`m.bar = 20; // Now m.foo is 10 and m.bar is 20`
  - Usually better to avoid introducing new properties in outside code and instead do entire definition in constructor

- **Methods**

- Properties whose values are functions

40

Java EE training: <http://courses.coreservlets.com>

## Objects: Example (Circle Class)

```
function Circle(radius) {
 this.radius = radius;

 this.getArea =
 function() {
 return(Math.PI * this.radius * this.radius);
 };
}

var c = new Circle(10);
c.getArea(); // Returns 314.1592...
```

41

Java EE training: <http://courses.coreservlets.com>

# The prototype Property

- **In previous example**
  - Every new Circle got its own copy of radius
    - Fine, since radius has per-Circle data
  - Every new Circle got its own copy of getArea function
    - Wasteful (if many Circles), since function definition never changes
- **Class-level properties**
  - `Classname.prototype.propertyName = value;`
- **Methods**
  - `Classname.prototype.methodName = function() {...};`
    - Just a special case of class-level properties
  - This is legal anywhere, but it is best to do it in constructor

42

Java EE training: <http://courses.coreservlets.com>

## Objects: Example (Updated Circle Class)

```
function Circle(radius) {
 this.radius = radius;

 Circle.prototype.getArea =
 function() {
 return(Math.PI * this.radius * this.radius);
 };
}

var c = new Circle(10);
c.getArea(); // Returns 3.141592...
```

43

Java EE training: <http://courses.coreservlets.com>

# Static Methods

- **Idea**

- Several related functions that do not use object properties
- You want to group them together and call them with `Utils.func1`, `Utils.func2`, etc.
  - Grouping is a syntactic convenience. Not real methods.
- Very similar to static methods in Java

- **Syntax**

- Assign functions to properties of an object, but do not define a constructor. E.g.,
  - ```
var Utils = new Object(); // Or function Utils() {}
  Utils.foo = function(a, b) { ... }
  Utils.bar = function(c) {
    var x = Utils.foo(val1, val2);
    var y = Utils.bar(val3);
```

44

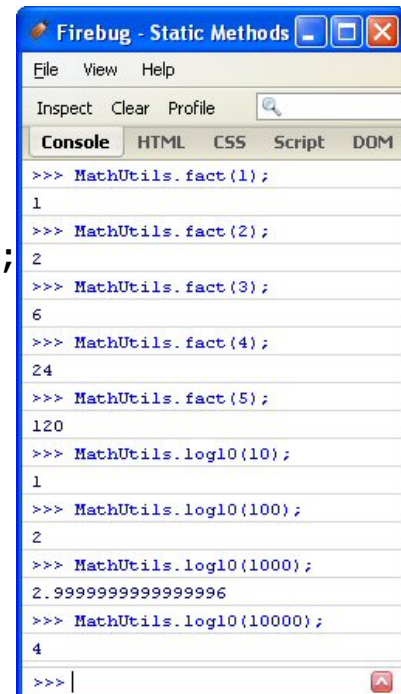
Java EE training: <http://courses.coreservlets.com>

Static Methods: Example (Code)

```
var MathUtils = new Object();

MathUtils.fact = function(n) {
  if (n <= 1) {
    return(1);
  } else {
    return(n * MathUtils.fact(n-1));
  }
}

MathUtils.log10 = function(x) {
  return(Math.log(x)/Math.log(10));
}
```



45

Java EE training: <http://courses.coreservlets.com>

JSON (JavaScript Object Notation)

- **Idea**

- A simple textual representation of JavaScript objects
- Main applications
 - One-time-use objects (rather than reusable classes)
 - Objects received via strings

- **Directly in JavaScript**

- `var someObject =`
 `{ 'property1': value1, // Single or double quotes`
 `'property2': value2,`
 `... };`

- **In a string (e.g., when coming in on network)**

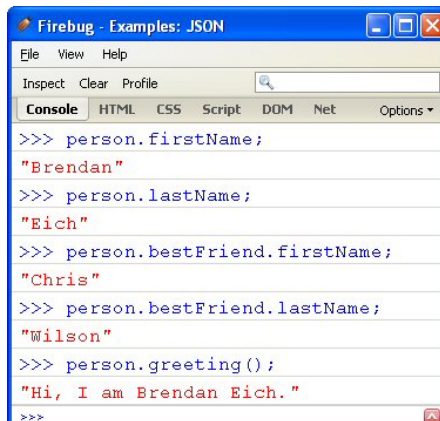
- Surround object representation in parens
- Pass to the builtin "eval" function

46

Java EE training: <http://courses.coreservlets.com>

JSON: Example

```
var person =  
  { 'firstName': 'Brendan',  
    'lastName': 'Eich',  
    'bestFriend': { 'firstName': 'Chris',  
                   'lastName': 'Wilson' },  
    'greeting': function() {  
      return("Hi, I am " + this.firstName +  
            " " + this.lastName + ".");  
    }  
  };  
};
```



47

g: <http://courses.coreservlets.com>

Other Object Tricks

- **The instanceof operator**
 - Determines if lhs is a member of class on rhs
 - `if (blah instanceof Array) {
doSomethingWith(blah.length);
}`
- **The typeof operator**
 - Returns direct type of operand, as a String
 - "number", "string", "boolean", "object", "function", or "undefined".
 - Arrays and null both return "object"
- **Adding methods to builtin classes**

```
String.prototype.describeLength =  
function() { return("My length is " + this.length); };  
"Any Random String".describeLength();
```
- **eval**
 - Takes a String representing *any* JavaScript and runs it
 - `eval("3 * 4 + Math.PI");` // Returns 15.141592

48

Java EE training: <http://courses.coreservlets.com>

© 2008 Marty Hall



Wrapup

Customized Java EE Training: <http://courses.coreservlets.com/>

Servlets, JSP, Struts, JSF/MyFaces/Facelets, Ajax, GWT, Java 5 or 6, etc. Spring/Hibernate coming soon.

Developed and taught by well-known author and developer. At public venues or onsite at *your* location.

Summary

- **Use Firebug for testing and debugging**
- **Bookmark references**
 - <http://www.w3schools.com/js/>
- **Embedding in browser**
 - `<script src="blah.js" type="test/javascript"></script>`
- **Basic syntax**
 - Mostly similar to Java
- **Functions**
 - Very different from Java. Passing functions around and making anonymous functions very important.
- **Objects**
 - Constructor also defines class. Use "this".
 - Not "real" OOP

50

Java EE training: <http://courses.coreservlets.com>

© 2008 Marty Hall



Questions?

Customized Java EE Training: <http://courses.coreservlets.com/>

Servlets, JSP, Struts, JSF/MyFaces/Facelets, Ajax, GWT, Java 5 or 6, etc. Spring/Hibernate coming soon.

Developed and taught by well-known author and developer. At public venues or onsite at *your* location.